Paperplane: zero to mastery

Docling, PDF Inspector, cloud AI, and local Ollama document intelligence

Table of Contents

1. Start

Double-click Paperplane.cmd on Windows or run ./Paperplane.sh on Linux. Each launcher installs only missing or out-of-date supported prerequisites and opens the multipage Streamlit app at http://127.0.0.1:8551. Once ready, later launches skip setup. Manual entrypoint:

```
uv run --locked --extra cpu python -m paperplane.model_store --prepare
uv run --locked --extra cpu streamlit run workspace_app.py --server.port=8551
```

The model command migrates existing Docling, RapidOCR, and PP-DocLayoutV3 caches when possible, then keeps the pinned set outside the checkout and virtual environment. Later launches reuse healthy weights offline after a manifest and size check. Ollama models stay in Ollama's separate store.

2. Parse

All four engine toggles start off. Choose Docling ADE, PDF Inspector ADE, Cloud AI ADE, or Ollama ADE. Optionally add cloud enhancement to a local engine. Upload up to 20 files, choose each one-based inclusive page range, then parse. Files run concurrently but remain context-isolated.

GLM-OCR, PaddleOCR-VL, and DeepSeek-OCR use a local CPU PP-DocLayoutV3 detector. Ollama recognizes each region with its family-native prompt, and Paperplane uses detector boxes for candidate grounding. RapidOCR contributes only exact final word-box alignment. DeepSeek retries one empty or transiently failed text crop. An isolated exhausted region is reported as a warning while successful regions remain; three consecutive failures stop the page and surface an actionable Ollama error.

All Parse controls sit below navigation in the sidebar. One main document selector drives Input preview, Output, Annotated PDF, Markdown, HTML, and JSON. Download strict ADE v2 JSON when you need the compatibility-shaped hierarchy; download Paperplane JSON for observed words, confidence state, provenance, warnings, and document relations. Standalone HTML is sanitized. The batch ZIP groups every successful document's available outputs and records failures in a versioned manifest without copying source uploads.

For PDFs, Input preview immediately follows the current inclusive page controls. Annotated PDF preview follows the range saved by the successful parse. These are display-only slices: the annotated-PDF download and batch ZIP keep the complete source PDF.

While processing, one batch-wide progress bar shows the current document, finalized page, or output-building stage with a percentage. The value measures completed work rather than remaining time, so native whole-document engines may advance in larger jumps. Completed failures still count, allowing every finished batch to reach 100%.

Uploads, engine settings, page ranges, and results remain in the browser session when you move between Parse, Organize, Jobs, and Cost. **New parse** clears the active Parse workspace but does not reset the session Cost ledger.

3. Organize

Organize runs Classify, Split, and Section over the grounded Parse response. Results retain source ranges and identify deterministic partials.

4. Understand grounding

Pages and blocks carry normalized boxes and half-open Unicode ranges. Text/marginalia use atomic line grounding; table cells carry row/column/span data. Native PDF words and RapidOCR word observations are emitted only when their text aligns exactly. IDs are zero-based and response-local in strict ADE v2 output.

5. Jobs, cost, and privacy

SQLite metadata and private artifacts live under %LOCALAPPDATA%\Paperplane for seven days. Jobs exposes status, cancellation state, per-job deletion, and clear-all. Credentials are never stored there. Cloud engines, including Agnes, transmit only selected pages. Agnes sends page PNGs inline instead of publishing them at public URLs. Ollama, Docling, PDF Inspector, and storage remain local.

Cost shows provider-reported input, cached-input, and output tokens for successful parses in the current browser session. It groups usage by the model that consumed the tokens, including separate Ollama and cloud-enhancement rows, then shows a total. Local and free models still show token usage at $0 API cost. Estimates use configured rates and are not provider invoices. **Stop and clear**, server restart, or session end resets this ledger; it is not written to retained job storage.

Paperplane is self-hosted, MIT-licensed software. Operators provide their own optional cloud API keys and are responsible for the files they process, permission to process them, provider terms and charges, compliance requirements, output review, and deletion of retained artifacts. The maintainers do not receive user documents or credentials through the project.

6. Follow the code

1. workspace_app.py — navigation.
2. streamlit_app.py and app_pages/ — UI flows, retained session state, and Cost.
3. paperplane/runtime.py — batch/provider composition.
4. paperplane/parser.py — range and page orchestration.
5. paperplane/model_store.py — permanent versioned Docling/RapidOCR/layout weights.
6. paperplane/ollama_document.py and ollama_ocr.py — Ollama discovery, layout regions,

prompts, and crop recognition.

7. paperplane/contracts.py and ade_contracts.py — internal and public contracts.
8. paperplane/ade_workflows.py — cited workflows.
9. paperplane/jobs.py — durable lifecycle.
10. paperplane/outputs.py — sanitized HTML and safe batch archive assembly.
11. document_intelligence.py, calibration.py, benchmark.py — semantic and evaluation

layers.

7. Verify

```
uv run ruff check paperplane app_pages tests streamlit_app.py workspace_app.py scripts
uv run pyright
uv run pytest -q
uv run python scripts/benchmark_report.py
```

The initial benchmark corpus is an integrity baseline, not an accuracy claim. Publish raw outputs, prompts, versions, costs, failures, and calibration provenance before publishing scores.